75.41 – Algoritmos y Programación II – Curso Ing. Calvo – **Parcial de Punteros**

Fecha: 5 de Octubre de 2009

Padrón:

Nombre:

1. Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
typedef int* Pint;

int main(){

  int *A, *C, *F;
  Pint *B;
  int H = 20;

  A = new int[3];

  for (int i = 0; i < 3; i++) {
      A[i] = H + i;
  }

  C = new int;
  (*C) = A[1];
  F = A + 2;

  cout << (*F) << (*C) << A[0] << endl;

  B = new Pint[3];
  B[0] = C;
  B[1] = F;
  B[2] = &H;

  cout << *B[1] << **B << *B[2] << endl;

  (*B[0]) = (*F) + 3;
  H++;
  A[2] = (*C) + 1;

  cout << *C << *B[1] <<  *B[2] << endl;

  F = C;
  C = A + 2;
  (**B) = A[0];
  A[2] = ** (B + 2);

  cout << H << (*C) << (*F) << endl;

  // liberar la memoria
  // ...

  return 0;
}
```

2. ¿Cuantas etapas tiene el algoritmo Hopfield? Indique el objetivo de cada una de ellas.

75.41 – Algoritmos y Programación II – Curso Ing. Calvo – **Parcial de Punteros**

Fecha: 5 de Octubre de 2009

Padrón:

Nombre:

1. Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
typedef int* Pint;

int main(){

  int *A, *C, *F;
  Pint *B;
  int H = 30;

  A = new int[3];

  for (int i = 0; i < 3; i++) {
      A[i] = H + i;
  }

  C = new int;
  (*C) = A[1];
  F = A + 2;

  cout << (*F) << (*C) << A[0] << endl;

  B = new Pint[3];
  B[0] = C;
  B[1] = F;
  B[2] = &H;

  cout << *B[1] << **B << *B[2] << endl;

  (*B[0]) = (*F) + 3;
  H++;
  A[2] = (*C) + 1;

  cout << *C << *B[1] <<  *B[2] << endl;

  F = C;
  C = A + 2;
  (**B) = A[0];
  A[2] = ** (B + 2);

  cout << H << (*C) << (*F) << endl;

  // liberar la memoria
  // ...

  return 0;
}
```

2. ¿Cuantas etapas tiene el algoritmo Hopfield? Indique el objetivo de cada una de ellas.